The `ibpy` Interface

A reference to `interface/ibpy.py`

The single boundary between this animation package and Blender's `bpy` API

`ibpy.py` encapsulates almost all direct calls into Blender. Every scene in the `video_*` folders talks to Blender through *this module, never to `bpy` directly. The practical payoff: when the Blender API changes between versions, the fix lives in one file; and the module doubles as a searchable catalogue of "how do I do X in this package". This document mirrors the section ordering that the source file itself now uses — search the source for `=== <SECTION NAME>` to jump straight to the corresponding code.*

Generated for the reorganised `ibpy.py` (375 functions, 25 sections).

Contents

1 How to read this document

The module is a flat collection of free functions; you import it and call them with a Blender object (conventionally a `bob` — "Blender object") plus parameters:

```
from interface import ibpy
# or:  from interface.ibpy import set_camera_location, camera_zoom
```

A few conventions recur throughout:

`bob`
the target object. Most functions accept either a real `bpy` object or one of this package's wrapper objects and call `get_obj` internally to normalise it.

`begin_time` / `transition_time`
animation timing in *seconds*. They are converted to frames with the module-level `FRAME_RATE`. Many "`change_*`" functions return `begin_time+transition_time`, so you can chain them: `t0=ibpy.change_default_value(...,begin_time=t0)`.

`begin_frame` / `frame_duration`
the older, frame-based variants of the same idea (integer frames).

`slot`
index into an object's material slots (default `0`).

The sections below follow the file's own ordering. Within each section the most-used functions are documented with runnable usage snippets taken from the `video_*` scenes; the remaining helpers are summarised so the section is complete.

2 Context, scene, frames & file I/O

Entry points to Blender's global state and to saving/loading.

`get_context()`, `get_scene()`, `get_layout()`
thin accessors for `bpy.context`, the active scene, and the "Layout" screen. Used internally far more than in scene code.

`set_frame(frame)`, `get_frame()`, `start_frame()`, `end_frame()`
move/query the playhead and the scene's frame range.

`close(x,y,precision=EPS)`
floating-point "almost equal" test used by geometry helpers.

`save(filename)`, `load(filepath)`, `load_file(filename)`, `append(filename)`
persistence. `append` pulls assets from an external `.blend`.

`set_cursor_location(position)`, `cursor_to_origin()`
drive the 3D cursor (which several "set origin" operations rely on).

`register_class(cls)`
register a custom `bpy` class.

```
# advance the timeline and persist the result
ibpy.set_frame(0)
ibpy.save("/tmp/scene_v1.blend")
```

3 Object lookup, creation, copying & deletion

`get_obj(bob)`
the workhorse: returns the underlying `bpy` object for a wrapper, a name, or a `bpy` object (idempotent). Almost every other function calls it first.

`get_obj_from_name(name)`, `get_objects_from_name(name)`
look up by (sub)string of the object name.

rename(bob,name)
rename an object.

create(mesh,name,location)
wrap a mesh datablock into a new scene object at a location.

add_empty(kwargs)**
add an Empty (handy as a camera target or parent pivot).

copy(bob), duplicate(bob)
shallow copy / full duplicate.

delete(obj)
remove an object from the scene.

get_child_with_name(bob,child_name,starts_with=False)
find a specific child in a hierarchy.

```
target = ibpy.add_empty(location=[0, 0, -6], name="CameraTarget")
clone  = ibpy.duplicate(some_object)
ibpy.rename(clone, "Clone.001")
```

4 Mesh primitives & mesh creation

Convenience wrappers around Blender's primitive operators, plus the all-important `create_mesh` for arbitrary geometry.

create_mesh(vertices,edges=[],faces=[],name='mesh',orient_faces=False)
build a mesh datablock from raw vertex/edge/face lists. The most general entry point; everything else is a shortcut.

add_sphere, add_cube, add_cone, add_cylinder, add_torus, add_circle, add_plane
the usual primitives; all take `location`, `scale`, and `smooth` where meaningful.

add_reference_image(name,kwargs)**
add an image-empty for modelling reference.

create_vertex_group(bob,vertex_indices,name), add_vertex_group(bob,indices,name,weight_function)
define weighted vertex groups (used by modifiers and deformations).

```
# build a polyhedron from explicit data and hand it to a wrapper object
mesh = ibpy.create_mesh(vertices, faces=faces)
kwargs_red = {"mesh": mesh, "color": "trunc_icosidodeca"}

# a smooth-shaded reference sphere
ball = ibpy.add_sphere(radius=1, location=(0, 0, 0), resolution=5,
                       smooth=True)
```

5 Mesh editing, conversion & analysis

get_vertex_locations(bob), get_vertices(bob)
read back vertex coordinates / vertices.

smooth_mesh(bob), shade_smooth(auto=True)
smoothing.

analyse_mesh(bob)
print a structural report (vertex / edge / face counts) — a debugging aid.

apply(bob,location=True,rotation=True,scale=True)
apply transforms into the mesh data (bakes them into vertex coordinates). `apply_scale(bob)` is the scale-only shortcut.

convert_to_mesh(obj,apply_transform=False)
convert a curve / text / other object into a mesh.

`separate(bob,type='SELECTED')`
split selected geometry into a new object.

`select_faces(faces,normal)`, `stretch_uv_map_to_faces(bob,normal)`
face selection and UV stretching (used for image-on-geometry materials).

`add_attribute(bob,name,attribute,type='FLOAT',domain='POINT')`
add a custom mesh attribute (read later by geometry nodes).

```
ibpy.apply(bob, location=False, rotation=True, scale=True)
verts = ibpy.get_vertex_locations(bob)     # list of Vectors
ibpy.add_attribute(bob, name="charge", attribute=charges, type="FLOAT")
```

6 Selection & edit/object mode

A complete set of (de)selection helpers plus mode switching. They wrap the fiddly `bpy.ops` selection state machine.

`set_edit_mode()`, `set_object_mode()`
toggle interaction mode.

`set_active(obj)`, `get_active()`
manage the active object.

`select(obj,deselect_others=True)`, `un_select(obj)`, `deselect_all()`
basic selection.

`select_recursively(obj)`, `select_all_deep(bobs)`, `deep_select(obj)`
select an object together with its descendants.

`set_select(obj,value=True)`, `set_several_select(objects,value=True)`
bulk set selection flags without operators.

```
ibpy.deselect_all()
ibpy.select(bob)                  # also makes it active, deselecting others
ibpy.set_edit_mode()
# ... operate ...
ibpy.set_object_mode()
```

7 Collections & linking

`make_new_collection(name,hide_render=False,hide_viewport=False)`
create a collection.

`get_collection(name)`, `get_collection_name(obj)`, `remove_collection(collection)`
manage collections.

`link(obj,collection=None,recursively=True)`, `recursive_link(obj,collection)`, `un_link(obj,collection)`
link/unlink objects into a collection (objects must be linked to appear in the scene). `is_linked(obj)` tests membership.

`to_collection(collection)`, `collection_to_string(collection)`
utilities used by the importer.

```
coll = ibpy.make_new_collection("Particles", hide_render=False)
ibpy.link(particle_obj, collection="Particles", recursively=True)
```

8 Visibility (hide / unhide) & fading

Two flavours of "make it disappear": *hard* hide (toggles `hide_render`) and *soft* fade (animates material alpha). The `*_rec` / `recursive_*` variants walk the child hierarchy.

`hide(bob,begin_time=0)`, `unhide(bob,begin_time=0)`
keyframe render visibility at a time.

`hide_rec`, `unhide_rec`, `hide_frm_rec`, `unhide_frm_rec`
recursive / frame-based versions.

`is_hidden(obj)`, `set_hide(bob,hide,frame)`
query / set raw flags.

`fade_in(b_obj,frame,frame_duration)`, `fade_out(b_obj,frame,frame_duration,alpha=0,children=True)`
animate alpha up/down; recurse into children by default. `fade_out_quickly` and `recursive_fade_out` are variants.

`set_shadow(value)`, `set_shadow_of_object(b_object,shadow)`
toggle shadow casting.

```
ibpy.fade_in(label, frame=0, frame_duration=30)
# later: hide from render after 5 s
ibpy.hide(label, begin_time=5)
```

9 Lights & light probes

`add_point_light(**kwargs)`, `add_area_light(**kwargs)`, `add_spot_light(energy,**kwargs)`
add lights; kwargs include `location`, `radius`, `scale`, `energy`.

`set_sun_light(location,energy=2)`, `remove_sun_light()`
the scene sun.

`add_light_probe(**kwargs)`
add a reflection/irradiance probe.

`switch_on(bob,...)`, `switch_off(bob,...)`, `change_power(bob,from_value,to_value,...)`
animate a light's power; `get_energy_at_frame(bob,frame)` reads it back.

`light_path_settings(total=12,diffuse=4,glossy=4,transmission=12,volume=0,transparency=8)`
global Cycles light-path bounce limits — a common quality/perf knob.

```
ibpy.add_point_light(location=[4, -4, 6], energy=2000, radius=0.5)
ibpy.light_path_settings(total=12, transmission=12, transparency=8)
ibpy.change_power(lamp, from_value=0, to_value=5000,
                  begin_frame=0, frame_duration=30)
```

10 Camera setup & animation

The most frequently used section. A scene typically: adds/places a camera, points it at a target Empty, sets the lens, then animates zoom/move.

`add_camera(location,rotation)`, `get_camera()`
create / get the camera.

`set_camera_location(location,frame=0)`, `set_camera_rotation(rotation)`
place it.

`set_camera_lens(lens=50,clip_end=1000)`
focal length and far clip.

`set_camera_view_to(target,rotation_euler=[0,0,0],targetZ=False,up_axis='UP_Y')`
aim the camera at a target with a Track-To constraint — the standard way to frame a subject.

camera_zoom(lens,begin_time,transition_time)
animate the focal length (dolly-zoom feel).

camera_move(shift,begin_time,transition_time)
translate the camera over time.

camera_rotate_to(rotation_euler,...)
animate orientation.

set_camera_dof(focus_object,aperture_fstop=0.1)
depth of field.

set_camera_follow(target), set_camera_copy_location(target), camera_follow(target,initial_value,final_value,...), camera_change_track_influence(...), camera_change_follow_influence(...)
constraint-based camera rigs (follow a path, copy a location, blend influence in/out).

camera_alignment_euler(bob,camera_location), camera_alignment_quaternion(bob,camera_location,default)
compute the rotation that makes an object face the camera (billboarding).

```
# canonical opening shot (from video_4d_geometry)
camera_empty = ibpy.add_empty(location=[0, 0, -6])
ibpy.set_camera_view_to(camera_empty)
ibpy.set_camera_location(location=Vector([-30, 55, 55]))
ibpy.set_camera_lens(lens=50)

# animate a slow zoom-in over 2 s
ibpy.camera_zoom(lens=30, begin_time=0.1, transition_time=2)
# then track the action for the rest of the shot
ibpy.camera_move(shift=[0, 0, 4], begin_time=2, transition_time=6)
```

11 Constraints, tracking, follow paths & parenting

The general constraint machinery that the camera section builds on, plus object parenting.

add_constraint(b_object,type,name=None,kwargs)**
add any Blender constraint by type string.

set_track(bob,target), set_copy_location(bob,target), set_follow(bob,target,kwargs)**
add Track-To / Copy-Location / Follow-Path constraints.

follow(bob,target,initial_value,final_value,begin_time,transition_time)
animate movement along a follow-path target.

set_track_influence, set_follow_influence, change_follow_influence
blend constraint influence over time.

set_parent(b_obj,parent), get_parent(bob), get_oldest_parent(bob), clear_parent(bob)
object parenting.

set_vertex_parent(parent,child,vertex_index)
parent a child to a single vertex (used for the "chain of arrows" constructions).

```
ibpy.set_parent(child_obj, parent_obj)        # rigid hierarchy
ibpy.set_track(arrow, target=tip_empty)       # always point at the tip
```

12 Transform animation (move / rotate / scale / grow / shrink)

Keyframed transforms. Frame-based functions take `begin_frame,frame_duration`; the `shrink`/`grow` pair has both a time-based and a scale-based overload (the later definition wins, see the note at the end).

move(b_obj,direction,begin_frame,frame_duration), move_to(b_obj,target,begin_frame,frame_duration,global_system=False), move_fast_from_to(b_obj,start,end,...)
translate by a delta / to an absolute point.

rotate_by(b_obj,rotation_euler), rotate_to(b_obj,begin_frame,frame_duration,pivot,interpolation)
rotation (instant / animated about a pivot).

grow(b_obj,scale,begin_frame,frame_duration,initial_scale=0,modus='from_center'), shrink(b_obj,scale,...)
scale-in / scale-out animation.

grow_from(b_obj,pivot,...), shrink_from(b_obj,pivot,...), rescale(b_obj,re_scale,...)
scale about an explicit pivot.

set_location(b_obj,location), set_origin(bob,type='ORIGIN_GEOMETRY'), set_pivot(obj,origin), transform_into_world(b_obj,location)
non-animated placement and origin control.

```
# scale an object up from nothing over 0.5 s (15 frames @ 30 fps)
ibpy.grow(bob, scale=1, begin_frame=0, frame_duration=15,
          initial_scale=0, modus="from_center")

ibpy.set_origin(bob, type="ORIGIN_GEOMETRY")   # center the pivot
```

13 Materials: assignment, color, alpha & emission

The largest section (37 functions). Covers attaching materials to objects and animating their scalar/colour properties.

add_material(bob,material), set_material(bob,material,slot=0), append_materials(bobj,materials)
attach materials to slots. `get_material(material,**kwargs)` resolves a material by name or spec; `get_materials(bob)`, `get_material_of(bob,slot)` read them back.

assign_material_to_faces(bob,material_index,normal), set_color_to_faces(bob,face2color_function), create_color_map_for_mesh(bob,colors,name), set_vertex_colors(bob,colors)
per-face / per-vertex colouring.

change_alpha(b_obj,frame,frame_duration,...), change_alpha_of_material(mat,from_value,to_value,begin_time,transition_time), set_alpha_for_material(material,alpha), set_alpha_and_keyframe(obj,value,frame,slot)
animate transparency. `get_alpha_at_current_keyframe(obj,frame,slot)` reads it.

set_emission, set_emission_color, change_emission, change_emission_to, change_emission_by_name, change_emission_of_material
drive emission strength / colour.

set_metallic_for_material, set_roughness_for_material, set_transmission_for_material, set_emission_strength_for_material
one-shot PBR property setters.

change_brightness, change_color, mix_color, shader_value, adjust_mixer, set_mixer
colour-mix and value-node animation. `get_alpha_mixer`, `change_material_properties`, `get_new_material` are construction helpers.

```
ibpy.set_material(bob, my_material, slot=0)

# fade a material to fully transparent over 1 s
ibpy.change_alpha_of_material(mat, from_value=1, to_value=0,
                              begin_time=t0, transition_time=1)

# pulse the emission of slot 0
ibpy.change_emission(bob, from_value=0, to_value=5,
                     slot=0, begin_frame=0, frame_duration=30)
```

14 Shader / geometry node access & default-value drivers

Find nodes inside a material or node group, and animate their socket default values — the backbone of procedural animation in this package.

`get_node_tree(name,type='GeometryNodeTree')`
create/fetch a node tree.

`get_node_with_label(bob,label)`, `get_node_from_tree(tree,label)`, `get_node_from_shader(shader,label)`, `get_all_nodes_from_shader(shader,label)`, `get_shader_node_from_material(material,label)`, `get_nodes_of_material(bob,name_part,slot)`, `get_node_of_material(material,label)`
locate nodes by label.

`change_default_value(slot,from_value,to_value,begin_time,transition_time)`
the single most useful driver: keyframe a socket's `default_value` from one number to another. Returns the end time for chaining.

`set_default_value`, `change_default_boolean`, `set_default_boolean`, `change_default_integer`, `change_default_vector`, `change_default_rotation`, `change_default_quaternion`, `change_default_material`, `change_value`
type-specific variants for each socket kind.

`change_shader_value(b_object,node,input,initial_value,final_value,frame,frame_duration)`, `change_mixer(mixer,...)`
animate a specific node input / a mix factor.

`get_default_value_at_frame(socket,frm)`, `get_value_at_frame(function,frm)`, `get_parameter_at_frame(function,frm)`
sample animated values.

`add_item_to_switch(switch_node,idx,socket_or_value,tree)`, `remove_unlinked_nodes(node_tree)`
node-tree editing utilities.

```
# chain several socket animations, threading the time t0 through:
t0 = 0
t0 = 0.5 + ibpy.change_default_value(sliders[-1], from_value=0,
                                     to_value=1, begin_time=t0,
                                     transition_time=1)
t0 = ibpy.change_default_value(ra, from_value=1, to_value=0,
                               begin_time=t0, transition_time=1)
```

15 Procedural material & shader builders, textures

Functions that *construct* entire shader node graphs — the domain-specific materials used across the videos (Mandelbrot colouring, hue maps, magnet fields, gradients, image/movie textures).

`customize_material(material,**kwargs)`
the central material customiser; pass `override_material` to replace special materials.

`make_mandelbrot_material`, `make_hue_material`, `make_iteration_material`, `make_magnet_material`, `make_gradient_material`, `make_dashed_material`, `make_alpha_frame`
ready-made procedural materials.

`make_coarse_grained_group`, `make_phase2color_group`, `make_vector_interpolation`, `make_mandel_iteration`
reusable node sub-groups consumed by the materials above.

`make_image__stretched_over_geometry(src,v_min,v_max)`, `add_image_texture(bob,image,...)`, `set_up_material_for_alpha_masking(bob)`, `create_image_mixing_find_previous_mixer(material,image_path)`
image-texture pipeline (incl. alpha masking).

`set_movie_to_material(bob,src,duration,...)`, `set_movie_start(bob,begin_frame)`
map a video clip onto a surface.

`create_color_mixing(material,color1,color2)`, `create_color_mixing_find_previous_color(material,color)`, `create_shader_from_script(material,osl_script,...)`, `create_voronoi_shader(ma`

terial,colors,...), create_shader_from_function(material,hue_functions,...)
colour mixers and OSL/Voronoi/RPN shader generators.

```
mat = ibpy.make_hue_material()
ibpy.set_material(plane, mat)
ibpy.add_image_texture(plane, image="diagram.png",
                       begin_time=0, transition_time=1)
```

16 Node-group & RPN function builders

Generic machinery that turns a mathematical formula in *reverse Polish notation* into a Shader/Geometry node group. These power `make_function`-style construction throughout the package.

create_group_from_scalar_function(nodes,functions,parameters=[],...), create_group_from_vector_function(nodes,functions,...)
build a node group computing scalar / vector RPN formulae.

create_shader_group_from_function(nodes,function,parameters,inputType,outputType,name)
shader-tree variant.

create_iterator_group(nodes,functions,parameters,iterations,name)
nest a function group `iterations` times (fractal / iterative maps).

if_node(nodes,bool_function,parameters,...)
a mix node switching between two inputs based on a boolean RPN expression.

make_new_socket(tree,name,io='INPUT',type='NodeSocketFloat')
add a typed socket to a group interface (used everywhere groups are built).

build_group_component, build_scalar_group_component, build_group, create_part, build_recursively
the recursive RPN-to-nodes compiler internals. `build_group` is deprecated in favour of `build_group_component`.

```
# add a vector input socket to a freshly created group tree
ibpy.make_new_socket(tree, name="position", io="INPUT",
                     type="NodeSocketVector")
```

17 Geometry-nodes modifier access & sockets

Reach into a geometry-nodes modifier to read its nodes and push values into its exposed sockets.

get_geometry_nodes_modifier(bob)
return the geometry-nodes modifier on an object. *(Defined twice; the second definition — the documented "extract modifier from bob" one — wins.)*

get_geometry_node_from_modifier(modifier,label)
fetch an inner node by label, e.g. to drive it later.

get_socket_names_from_modifier(modifier)
list the sockets that receive custom values during setup. *(Also defined twice; last wins.)*

set_socket_data_for_geometry_node_modifier(bob,socket_data)
bulk-assign exposed-socket values.

get_material_from_modifier(modifier,label)
pull a material generated inside the modifier.

```
mod        = ibpy.get_geometry_nodes_modifier(bob)
csv_node   = ibpy.get_geometry_node_from_modifier(mod, "Import CSV")
counter    = ibpy.get_geometry_node_from_modifier(mod, "Counter")
ibpy.change_default_value(counter.inputs["Value"], from_value=0,
                          to_value=100, begin_time=0, transition_time=4)
```

18 Volume scatter & absorption

set_volume_scatter(bob,value,begin_time=0), change_volume_scatter(bob,final_value,begin_time,transition_time)
animate volumetric scattering on an object.

set_volume_absorption(bob,value,begin_time=0), change_volume_absorption(bob,final_value,...)
volumetric absorption.

set_volume_scatter_of_material(material,value), set_volume_absorption_of_material(material,value)
material-level setters.

set_volume_scatter_background(density=1)
world-volume fog.

```
ibpy.change_volume_scatter(cloud, final_value=0.4,
                           begin_time=t0, transition_time=2)
```

19 World, background, HDRI, render & compositor

Render-engine configuration, the world/HDRI environment, and post-render compositor effects. `set_render_engine` and `set_hdri_background` are among the most-called functions in the whole package.

set_render_engine(engine='CYCLES',transparent=False,motion_blur=False,denoising=False,resolution_percentage=100,frame_start=0,taa_render_samples=1024,...)
one call to configure the render pipeline.

set_hdri_background(filename,ext='exr',simple=False,transparent=False,strength=0.2,rotation_euler=None,reflections=True,...)
load an HDRI as the world background.

set_hdri_strength(strength,begin_time,transition_time), change_hdri_strength(start,end,...), hdri_background_rotate(rotation_euler,...)
animate the HDRI.

get_background(), get_sky(), animate_sky_background(), dim_background(value,...), set_volume_scatter_background(density)
other world controls.

create_composition(denoising=None), empty_blender_view3d(), set_taa_render_samples(taa_render_samples=64,begin_frame=0), set_exposure(value)
compositor / viewport / exposure.

animate_glare(glare,kwargs), change_glow_threshold(...), change_glow_size(...), get_image(src)**
glare/glow compositor effects.

```
# canonical render + world setup (from video_4d_geometry)
ibpy.set_hdri_background("kloofendal_misty_morning_puresky_4k", 'exr',
                         simple=True, transparent=True,
                         rotation_euler=Vector())
t0 = ibpy.set_hdri_strength(1, begin_time=0, transition_time=0)
ibpy.set_render_engine(denoising=False, transparent=True, frame_start=1,
                       resolution_percentage=100, engine="CYCLES",
                       taa_render_samples=64, motion_blur=True)
```

20 Curves, splines & bevel

Bezier-curve construction and the bevel/extrude controls that turn a curve into a renderable tube, plus "grow along the curve" animation.

`import_curve(path)`, `new_curve_object(name,curve)`, `get_new_curve(name,num_points,data,**kwargs)`
curve creation / import.

`add_bezier_spline(splines,num_points=2,data,cyclic=True)`, `add_points_to_spline(spline,total_points,closed_loop)`, `set_bezier_point_of_curve(curve,index,position,handle_left,handle_right)`, `reset_bezier_point(...)`
build / edit splines and control points.

`get_splines(obj)`, `get_splines_of_curve(b_obj)`, `get_bezier_points(spline)`, `get_curve_for_object(obj)`, `get_all_curves()`, `merge_splines(b_objects)`, `separate_pieces(data)`, `get_spline_bounding_box(b_obj)`
inspect / combine curve data.

`set_bevel(bob,depth,caps=True,res=4)`, `change_bevel(bob,old_depth,new_depth,...)`, `set_extrude(bob,extrude)`, `set_bevel_factor_mapping(bob,...)`
give a curve thickness.

`grow_curve(bob,appear_frame,duration,...)`, `shrink_curve(bob,appear_frame,duration,inverted=False)`, `set_curve_range(bob,factor,...)`, `set_curve_full_range(bob,factor1,factor2,...)`, `set_use_path(bob,is_used)`
animate a curve drawing itself in / out (via bevel-factor start/end).

```
ibpy.set_bevel(curve_obj, depth=0.02, caps=True, res=6)
# draw the curve on, left to right, over 1.5 s
ibpy.grow_curve(curve_obj, appear_frame=0, duration=45)
```

21 Shape keys & morphing

Animate between mesh shapes via shape keys — the basis of the "morph one object into another" transitions.

`add_shape_key(bobj,name,previous=None,following=None,relative=True)`
add a shape key.

`create_shape_key_from_transformation(bob,old_sk,index,transformation)`, `create_shape_key_from_index_transformation(bob,old_sk,index,transformation)`
derive a new shape by transforming an existing one.

`keyframe_shape_key(bob,sk_name,frame)`, `set_shape_key_to_value(bob,sk_name,value,frame)`, `set_shape_key_eval_time(bob,time,frame)`
keyframe shape-key values / evaluation time.

`morph_to_next_shape`, `morph_to_next_shape2`, `morph_to_next_shape_with_pause`, `morph_to_previous_shape`, `morph_to_first_shape`, `set_to_first_shape`, `set_to_last_shape`, `zero_all_shape_keys`
high-level morph sequencing (the `*2` variant zeroes all but the current key).

```
ibpy.add_shape_key(bob, "target")
ibpy.morph_to_next_shape(bob, current_shape_index=0,
                         appear_frame=0, frame_duration=30)
```

22 Modifiers

Add, configure, animate and apply Blender modifiers (incl. geometry nodes).

`add_mesh_modifier(bob,**kwargs)`
the generic, preferred entry point (dispatches on a `type` kwarg). `replace_mesh_modifier` swaps one out.

`add_modifier(b_obj,type,name=None)`, `add_modifier_recursive(obj,type,name)`
low-level add by type.

`add_sub_division_surface_modifier(b_obj,level=2,...)`, `add_solidify_modifier(b_obj,thickness=0.1)`, `add_bevel_modifier(b_obj,width=0.1)`
common specific modifiers.

change_solidifier_thickness(bob,from_value,to_value,...), change_modifier_attribute(bob,mod_name,attr_name,start,end,...)
animate modifier parameters.

apply_modifier(bob,type=None), apply_modifiers(bob)
bake modifiers into the mesh.

```
ibpy.add_solidify_modifier(shell, thickness=0.05)
ibpy.change_modifier_attribute(bob, "Solidify", "thickness",
                               start=0, end=0.1,
                               begin_frame=0, frame_duration=30)
ibpy.apply_modifiers(bob)
```

23 Keyframes, F-curves & animation interpolation

The low-level keyframe layer that all the `change_*` functions sit on, plus tools to make animations linear and to debug F-curves.

insert_keyframe(bob,data_path,frame)
insert a keyframe on any data path.

set_bevel_factor_and_keyframe(data,value,frame), set_bevel_factor_start_and_end_keyframe(data,start,end,frame), set_bevel_factor_start_keyframe(data,start,frame)
curve-draw keyframing primitives used by `grow_curve`.

set_linear_fcurves(bob), set_linear_action(bob,data_path_part), set_linear_action_full(bob), set_linear_action_modifier(bob), set_bezier_action_modifier(bob), set_linear_fcurves_for_nodes(node), make_animations_linear(thing,data_paths,extrapolate=False)
force linear (constant-speed) interpolation on selected channels.

clear_animation_data(bob)
wipe all animation from an object.

value_of_action_fcurve_at_frame(action,fcurve,frm), print_actions(), print_fcurves_of_action(action)
inspection / debugging helpers ("first used in `video_apollonian`").

```
ibpy.insert_keyframe(bob, "location", frame=0)
ibpy.make_animations_linear(bob, data_paths=["location"])
```

24 Rigid body, forces & physics simulation

make_rigid_body(bob,dynamic=True,kinematic=False,friction=0.1,bounciness=0.5,...), make_rigid_bodies(bob_list,...)
turn one or many objects into rigid bodies.

add_rigid_body(bob,kwargs), key_frame_rigid_body_properties(bob,type='dynamic',value=True,begin_time=0)**
lower-level rigid-body control and keyframing the dynamic/kinematic switch.

add_force(kwargs), add_wind(**kwargs), add_turbulence(**kwargs), decay_force(bob,initial_strength,final_strength=0,...)**
force fields.

set_simulation(begin_time=0,transition_time=250/FRAME_RATE)
run/scope the physics simulation over a time window.

```
for child in pieces:
    ibpy.make_rigid_body(child, type="ACTIVE",
                         collision_shape="MESH", friction=0.5)
```

```
ibpy.add_force(location=[0, 0, 0], strength=-50)   # gravity well
ibpy.set_simulation(begin_time=0, transition_time=4)
```

25 Geometry queries: bounding box, centers, world location

Read-only spatial queries: where is an object, how big is it, where is a particular feature. Heavily used to position labels and arrows relative to existing geometry.

`get_bounding_box(bob)`, `get_bounding_box_for_letter(letter)`, `analyse_bound_box(bob)`, `get_dimension(bob,direction)`
extents and size.

`find_center_of_leftest_vertices(obj)` and its `rightest` / `lowest` / `highest` / `closest` / `furthest` siblings
the geometric centre of an object's extremal face/edge/vertex — ideal anchor points for annotations.

`get_world_location(b_obj)`, `get_world_location2(b_obj)`, `get_world_matrix(bob)`, `get_location(b_obj)`, `get_scale(b_obj)`, `get_rotation(b_obj)`
current transform.

`get_world_location_at_time(b_obj,time)`, `get_location_at_frame(b_obj,frame)`, `get_scale_at_frame(b_obj,frame)`, `get_rotation_at_frame(b_obj,frame)`, `get_rotation_quaternion_at_frame(b_obj,frame)`, `get_offset_factor_at_frame(b_obj,target,frame)`, `get_input_value_at_frame(b_obj,frame)`
sample the transform / values at a specific time — essential when one animation must start where another leaves off.

```
top = ibpy.find_center_of_highest_vertices(bob)   # anchor a label here
where = ibpy.get_world_location_at_time(mover, time=2.0)
```

26 Miscellaneous utilities

`to_vector(z)`
coerce a list/tuple into a `mathutils.Vector` (`to_vector([1,2,3])->Vector((1.0,2.0,3.0))`).

`vectorize(list)`
map a nested list to Vectors.

`diff(a,b)`
element-wise difference helper.

```
v = ibpy.to_vector([1, 2, 3])     # Vector((1.0, 2.0, 3.0))
```

27 Notes on duplicate definitions

Three names are defined twice in the source; Python keeps the *last* definition, and the reorganisation preserves that "last wins" ordering so behaviour is unchanged:

- `get_geometry_nodes_modifier` — the second (documented "extract modifier from bob") definition is the live one.
- `get_socket_names_from_modifier` — second definition wins.
- `shrink` — the scale-based overload (with a `scale` argument) wins over the earlier time-based one.

If you intend to call the *earlier* behaviour, rename one of the definitions; calling the name as-is always reaches the later one.

Generated from the reorganised `interface/ibpy.py`. Section order and titles match the in-file `===` banners; search the source for a banner to jump to the code.